Negativa Fácil

Atividade do Negócio

Autor: Bruno Salles
Mogi Das Cruzes, 07 de Julho de 2024

Sumário

1 **Introdução** 3
2 **Objetivo** 3
3 **Fluxo de Atividade** 3
3.1 Interface Gráfica do Usuário (GUI) 3
3.1.1 *Objetivo* 3
3.1.2 *Componentes:* 3
3.1.3 *Código*: 4
3.2 Automatização do Processo 6
3.2.1 *Objetivo:* 6
3.2.2 *Componentes:* 6
3.2.3 *Código*: 7
3.3 Consulta e Exportação de Dados 8
3.3.1 *Objetivo* 8
3.3.2 *Componentes:* 8
3.3.3 *Código*: 8
4 **Conclusão** 9

1 Introdução

Este documento descreve o processo de automação para negativação de dados utilizando a biblioteca Selenium e interface gráfica tkinter em Python.

2 Objetivo

O objetivo deste processo é automatizar o preenchimento de dados de negativação em um sistema web, utilizando informações de um arquivo Excel fornecido pelo usuário.

3 Fluxo de Atividade

3.1 Interface Gráfica do Usuário (GUI)

3.1.1 *Objetivo*

- Criar uma interface amigável para que o usuário possa inserir informações necessárias para a automação.

3.1.2 *Componentes:*

- Campos para inserção de login, senha, número do credor e caminho do arquivo Excel.
- Botões para procurar o arquivo, iniciar a automação e consultar dados.

3.1.3 *Código*:

```python
# Importando as bibliotecas necessárias
import tkinter as tk  # Biblioteca para GUI em Python
from tkinter import filedialog, messagebox  # Componentes específicos do tkinter para diálogos e mensagens
from tkinter import ttk  # Componentes adicionais do tkinter para widgets mais avançados
from PIL import Image, ImageTk  # Biblioteca para trabalhar com imagens
from selenium import webdriver  # Biblioteca Selenium para automação de navegador web
from selenium.webdriver.common.keys import Keys  # Componente do Selenium para interação com teclado
from selenium.webdriver.common.by import By  # Componente do Selenium para localização de elementos por estratégias
from selenium.webdriver.support.ui import WebDriverWait  # Componente do Selenium para esperas explícitas
from selenium.webdriver.support import expected_conditions as EC  # Componente do Selenium para condições esperadas
import pandas as pd  # Biblioteca pandas para manipulação de dados
import time  # Biblioteca para operações relacionadas ao tempo
import sqlite3  # Biblioteca SQLite para banco de dados embutido

# Definição da função principal para automatizar o processo
def automate_process():
    # Obtendo dados dos campos de entrada na interface
    login = login_entry.get()
    password = password_entry.get()
    creditor_number = creditor_number_entry.get()
    file_path = file_path_entry.get()

    # Verificando se todos os campos foram preenchidos
    if not login or not password or not creditor_number or not file_path:
        messagebox.showerror("Error", "Todos os campos devem ser preenchidos.")
        return

    # Verificando se o número do credor contém apenas números
    if not creditor_number.isdigit():
        messagebox.showerror("Error", "O número do credor deve conter apenas números.")
        return

    try:
        # Lendo o arquivo Excel especificado pelo usuário
        df = pd.read_excel(file_path)
    except FileNotFoundError:
        messagebox.showerror("Error", "Arquivo não encontrado.")
        return

    # Inicializando o driver do navegador Edge (Selenium)
    driver = webdriver.Edge()
    driver.get("https://www.serasaexperian.com.br/meus-produtos/login")
    driver.maximize_window()

    try:
        # Preenchendo campos de login e senha na página web
        login_field = WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.XPATH, "//input[@id='loginUser']"))
        )
        password_field = driver.find_element(By.XPATH, "//input[@id='loginPassword']")
        btn_acess = driver.find_element(By.XPATH, "//button[@id='loginFormSubmit']")

        login_field.send_keys(login)
        password_field.send_keys(password)
        btn_acess.click()

        time.sleep(5)  # Aguardando 5 segundos para carregamento da página
```

```python
        # Realizando ações após login
        tour = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, '//*[@id="mat-mdc-dialog-0"]/div/div/div/mat-dialog-content/div/div[2]/div/a'))
        )
        tour.click()

        rec_div = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, '//*[@id="prod-64adce65a625ed4687bde841"]/div/div[3]/button'))
        )
        rec_div.click()

        incluc_div = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, "//span[@class='sc-e17868fb-2 dlssDw']"))
        )
        incluc_div.click()

        time.sleep(3)  # Aguardando 3 segundos

        # Conectando ao banco de dados SQLite
        conn = sqlite3.connect('negativacao.db')
        c = conn.cursor()

        # Criando tabela se não existir
        c.execute('''CREATE TABLE IF NOT EXISTS registros
                     (CNPJ TEXT, Razao_Social TEXT, Valor REAL, Titulo TEXT, Status TEXT, Data_Hora TEXT)''')

        # Definindo função para automatizar negativação de registros
        def automat_negative(row):
            try:
                # Preenchendo formulário de negativação com dados do DataFrame
                credor = WebDriverWait(driver, 10).until(
                    EC.presence_of_element_located((By.XPATH, "//input[@id='creditor.documentNumber']"))
                )
                credor.clear()
                credor_number_str = str(creditor_number).zfill(7)
                credor.send_keys(credor_number_str)

                CNPJ = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.documentNumber']")))
                CNPJ.clear()

                # Verificando e ajustando o formato do CNPJ/CPF
                cnpj_cpf_str = str(row['CNPJ/CPF'])
                if len(cnpj_cpf_str) > 11 and len(cnpj_cpf_str) <= 13:
                    cnpj_cpf_str = cnpj_cpf_str.zfill(14)
                elif len(cnpj_cpf_str) < 11:
                    cnpj_cpf_str = cnpj_cpf_str.zfill(11)

                CNPJ.send_keys(cnpj_cpf_str)

                Data_oc = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='dueDate']")))
                Data_oc.clear()
                data_vencimento = row['Vencimento'].strftime("%d/%m/%Y")
                Data_oc.send_keys(data_vencimento)

                valor_div = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='value']")))
                valor_div.clear()
                valor_formatado = "{:.2f}".format(row['Valor']).replace('.', ',')
```

```python
                valor_div.send_keys(valor_formatado)

                natureza = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="categoryId"]/option[38]')))
                natureza.click()

                titulo = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='contractNumber']")))
                titulo.clear()

                documento = str(int(row['Documento'])).zfill(9)

                if row['Parcela'] and str(row['Parcela']).strip():  # Verifica se a coluna Parcela não está vazia
                    titulo.send_keys(f"{documento}"+f"/{row['Parcela']}")
                else:
                    titulo.send_keys(documento)

                rz_social = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.name']")))
                rz_social.clear()
                rz_social.send_keys(row['Nome cliente'])

                cep = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.zipCode']")))
                cep.clear()
                cep.send_keys(row['CEP'])

                time.sleep(3)  # Aguardando 3 segundos

                logradouro = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.addressLine']")))
                logradouro.clear()
                logradouro.send_keys(row['Endereco'])

                numero = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.number']")))
                numero.clear()
                numero.send_keys(str(row['Nro']))

                complem = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.complement']")))
                complem.clear()
                complem.send_keys(str(row['Complem']))

                bairro = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="debtor.address.district"]')))
                bairro.clear()
                bairro.send_keys(row['Bairro'])

                uf = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.state']")))
                uf.clear()
                uf.send_keys(row['UF'])

                cidade = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.city']")))
                cidade.clear()
                cidade.send_keys(row['Cidade'])

                try:
                    # Tentando enviar o formulário
                    btn_enviar_div = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="__next"]/main/div/form/div[9]/div/button')))
                    btn_enviar_div.click()
                    status = "Enviado"
```

```python
                except:
                    status = "Não Enviado"

                time.sleep(3)  # Aguardando 3 segundos
                btn_modal = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="modal"]/div[1]/div/div/div[2]/div/div/button[2]')))
                btn_modal.click()

            except Exception as e:
                if driver.find_elements(By.XPATH, '//*[@id="toast-notification-c38fb503-75e0-4a27-a737-22afe3841796"]'):
                    status = "Erro: Notificação de erro detectada"
                else:
                    status = f"Erro: {str(e)}"

            # Registrando a ação no banco de dados SQLite
            data_hora = time.strftime("%Y-%m-%d %H:%M:%S")
            c.execute("INSERT INTO registros (CNPJ, Razao_Social, Valor, Titulo, Status, Data_Hora) VALUES (?, ?, ?, ?, ?, ?)",
                      (str(row['CNPJ/CPF']), row['Nome cliente'], row['Valor'], f"{documento}/{row['Parcela']}", status, data_hora))
            conn.commit()

        # Iterando sobre cada linha do DataFrame para automação
        for index, row in df.iterrows():
            if pd.notna(row['CNPJ/CPF']):  # Verificando se CNPJ/CPF não é nulo
                automat_negative(row)  # Chamando função para automatizar a negativação
                time.sleep(3)  # Aguardando 3 segundos entre cada ação

        driver.close()  # Fechando o navegador Selenium
        conn.close()  # Fechando conexão com o banco de dados SQLite
        messagebox.showinfo("Informação", "Processo concluído.")  # Exibindo mensagem de conclusão

    except Exception as e:
        driver.quit()  # Encerrando navegador Selenium em caso de erro
        messagebox.showerror("Erro", f"Ocorreu um erro: {str(e)}")  # Exibindo mensagem de erro

# Função para navegar e selecionar arquivo Excel na interface
def browse_file():
    file_path = filedialog.askopenfilename(filetypes=[("Excel files", ".xls;.xlsx")])
    file_path_entry.delete(0, tk.END)  # Limpando o campo de entrada atual
    file_path_entry.insert(0, file_path)  # Inserindo o caminho do arquivo selecionado no campo de entrada

# Função para consultar e exibir dados do banco de dados SQLite na interface
def consultar_dados():
    conn = sqlite3.connect('negativacao.db')
    df = pd.read_sql_query("SELECT * FROM registros", conn)
    conn.close()

    # Limpando a visualização atual na árvore de dados
    for i in tree.get_children():
        tree.delete(i)

    # Inserindo novos dados na árvore de dados (TreeView) na interface
    for index, row in df.iterrows():
        tree.insert("", "end", values=(row['CNPJ'], row['Razao_Social'], row['Valor'], row['Titulo'], row['Status'], row['Data_Hora']))
```

```python

# Função para exportar dados do banco de dados SQLite para um arquivo Excel
def exportar_dados():
    conn = sqlite3.connect('negativacao.db')
    df = pd.read_sql_query("SELECT * FROM registros", conn)
    conn.close()

    # Solicitando ao usuário o local para salvar o arquivo Excel exportado
    file_path = filedialog.asksaveasfilename(defaultextension=".xlsx", filetypes=[("Excel files", "*.xlsx")])
    if file_path:
        df.to_excel(file_path, index=False)  # Exportando DataFrame para o arquivo Excel
        messagebox.showinfo("Informação", "Dados exportados com sucesso.")  # Exibindo mensagem de sucesso

# Configuração da interface gráfica tkinter
app = tk.Tk()
app.title("Negativa Fácil")
app.configure(bg='#ED1B24')  # Configurando cor de fundo da janela principal

# Carregando e exibindo logotipo na interface
logo_path = "Logo Vm 1.png"
logo_image = Image.open(logo_path)
logo_image = logo_image.resize((100, 50), Image.Resampling.LANCZOS)
logo_photo = ImageTk.PhotoImage(logo_image)
logo_label = tk.Label(app, image=logo_photo, bg='#ED1B24')
logo_label.grid(row=0, column=0, padx=10, pady=10, columnspan=3, sticky='w')

# Labels e campos de entrada para login, senha, número do credor e arquivo
tk.Label(app, text="Login:", bg='#ED1B24', fg='white').grid(row=1, column=0, padx=10, pady=10, sticky='w')
login_entry = tk.Entry(app)
login_entry.grid(row=1, column=0, padx=60, pady=10, sticky='w')

tk.Label(app, text="Senha:", bg='#ED1B24', fg='white').grid(row=2, column=0, padx=10, pady=10, sticky='w')
password_entry = tk.Entry(app, show="*")
password_entry.grid(row=2, column=0, padx=60, pady=10, sticky='w')

tk.Label(app, text="Número do Credor:", bg='#ED1B24', fg='white').grid(row=3, column=0, padx=10, pady=2, sticky='w')
creditor_number_entry = tk.Entry(app)
creditor_number_entry.grid(row=4, column=0, padx=11, pady=2, sticky='w')

tk.Label(app, text="Arquivo:", bg='#ED1B24', fg='white').grid(row=5, column=0, padx=10, pady=0, sticky='w')
file_path_entry = tk.Entry(app)
file_path_entry.grid(row=6, column=0, padx=11, pady=0, sticky='w')

# Botões para selecionar arquivo, executar processo, consultar e exportar dados
tk.Button(app, text="Procurar", command=browse_file).grid(row=6, column=0, padx=140, pady=1, sticky='w')
tk.Button(app, text="Executar", command=automate_process).grid(row=7, column=0, columnspan=3, padx=10, pady=10, sticky='w')
tk.Button(app, text="Consultar", command=consultar_dados).grid(row=7, column=3, columnspan=3, padx=80, pady=0, sticky='e')

# Árvore de dados para exibir resultados da consulta
tree = ttk.Treeview(app, columns=("CNPJ", "Razao_Social", "Valor", "Titulo", "Status", "Data_Hora"), show="headings")
tree.heading("CNPJ", text="CNPJ")
tree.heading("Razao_Social", text="Razão Social")
tree.heading("Valor", text="Valor")
tree.heading("Titulo", text="Titulo")
tree.heading("Status", text="Status")
tree.heading("Data_Hora", text="Data e Hora")
tree.grid(row=8, column=0, columnspan=4, padx=10, pady=10)

# Botão para exportar dados exibidos na árvore de dados para um arquivo Excel
tk.Button(app, text="Exportar", command=exportar_dados).grid(row=7, column=0, columnspan=4, padx=10, pady=1, sticky='e')

# Iniciando a aplicação tkinter
app.mainloop()
```

3.2 Automatização do Processo

3.2.1 *Objetivo:*

- Automatizar o preenchimento dos dados de negativação utilizando Selenium WebDriver.

3.2.2 *Componentes:*

- Função automate_process():

3.2.3 *Código*:

```python
# Definição da função principal para automatizar o processo
def automate_process():
    # Obtendo dados dos campos de entrada na interface
    login = login_entry.get()
    password = password_entry.get()
    creditor_number = creditor_number_entry.get()
    file_path = file_path_entry.get()

    # Verificando se todos os campos foram preenchidos
    if not login or not password or not creditor_number or not file_path:
        messagebox.showerror("Error", "Todos os campos devem ser preenchidos.")
        return

    # Verificando se o número do credor contém apenas números
    if not creditor_number.isdigit():
        messagebox.showerror("Error", "O número do credor deve conter apenas números.")
        return

    try:
        # Lendo o arquivo Excel especificado pelo usuário
        df = pd.read_excel(file_path)
    except FileNotFoundError:
        messagebox.showerror("Error", "Arquivo não encontrado.")
        return

    # Inicializando o driver do navegador Edge (Selenium)
    driver = webdriver.Edge()
    driver.get("https://www.serasaexperian.com.br/meus-produtos/login")
    driver.maximize_window()

    try:
        # Preenchendo campos de login e senha na página web
        login_field = WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.XPATH, "//input[@id='loginUser']"))
        )
        password_field = driver.find_element(By.XPATH, "//input[@id='loginPassword']")
        btn_acess = driver.find_element(By.XPATH, "//button[@id='loginFormSubmit']")

        login_field.send_keys(login)
        password_field.send_keys(password)
        btn_acess.click()

        time.sleep(5)  # Aguardando 5 segundos para carregamento da página

        # Realizando ações após login
        tour = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, '//*[@id="mat-mdc-dialog-0"]/div/div/div/mat-dialog-content/div/div[2]/div/a'))
        )
        tour.click()

        rec_div = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, '//*[@id="prod-64adce65a625ed4687bde841"]/div/div[3]/button'))
        )
        rec_div.click()

        incluc_div = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, "//span[@class='sc-e17868fb-2 dlssDw']"))
        )
        incluc_div.click()

        time.sleep(3)  # Aguardando 3 segundos

        # Conectando ao banco de dados SQLite
        conn = sqlite3.connect('negativacao.db')
        c = conn.cursor()

        # Criando tabela se não existir
        c.execute('''CREATE TABLE IF NOT EXISTS registros
                     (CNPJ TEXT, Razao_Social TEXT, Valor REAL, Titulo TEXT, Status TEXT, Data_Hora TEXT)''')
```

```python
def automate_process():
        # Definindo função para automatizar negativação de registros
        def automat_negative(row):
            try:
                # Preenchendo formulário de negativação com dados do DataFrame
                credor = WebDriverWait(driver, 10).until(
                    EC.presence_of_element_located((By.XPATH, "//input[@id='creditor.documentNumber']"))
                )
                credor.clear()
                credor_number_str = str(creditor_number).zfill(7)
                credor.send_keys(credor_number_str)

                CNPJ = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.documentNumber']")))
                CNPJ.clear()

                # Verificando e ajustando o formato do CNPJ/CPF
                cnpj_cpf_str = str(row['CNPJ/CPF'])
                if len(cnpj_cpf_str) > 11 and len(cnpj_cpf_str) <= 13:
                    cnpj_cpf_str = cnpj_cpf_str.zfill(14)
                elif len(cnpj_cpf_str) < 11:
                    cnpj_cpf_str = cnpj_cpf_str.zfill(11)

                CNPJ.send_keys(cnpj_cpf_str)

                Data_oc = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='dueDate']")))
                Data_oc.clear()
                data_vencimento = row['Vencimento'].strftime("%d/%m/%Y")
                Data_oc.send_keys(data_vencimento)

                valor_div = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='value']")))
                valor_div.clear()
                valor_formatado = "{:.2f}".format(row['Valor']).replace('.', ',')
                valor_div.send_keys(valor_formatado)

                natureza = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="categoryId"]/option[38]')))
                natureza.click()

                titulo = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='contractNumber']")))
                titulo.clear()

                documento = str(int(row['Documento'])).zfill(9)

                if row['Parcela'] and str(row['Parcela']).strip():  # Verifica se a coluna Parcela não está vazia
                    titulo.send_keys(f"{documento}"+f"/{row['Parcela']}")
                else:
                    titulo.send_keys(documento)

                rz_social = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.name']")))
                rz_social.clear()
                rz_social.send_keys(row['Nome cliente'])

                cep = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.zipCode']")))
                cep.clear()
                cep.send_keys(row['CEP'])

                time.sleep(3)  # Aguardando 3 segundos

                logradouro = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.addressLine']")))
                logradouro.clear()
                logradouro.send_keys(row['Endereco'])

                numero = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.number']")))
                numero.clear()
                numero.send_keys(str(row['Nro']))

                complem = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.complement']")))
                complem.clear()
                complem.send_keys(str(row['Complem']))
```

```python

            bairro = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="debtor.address.district"]')))
            bairro.clear()
            bairro.send_keys(row['Bairro'])

            uf = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.state']")))
            uf.clear()
            uf.send_keys(row['UF'])

            cidade = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, "//input[@id='debtor.address.city']")))
            cidade.clear()
            cidade.send_keys(row['Cidade'])

            try:
                # Tentando enviar o formulário
                btn_enviar_div = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="__next"]/main/div/form/div[9]/div/button')))
                btn_enviar_div.click()
                status = "Enviado"
            except:
                status = "Não Enviado"

            time.sleep(3)  # Aguardando 3 segundos
            btn_modal = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.XPATH, '//*[@id="modal"]/div[1]/div/div/div[2]/div/div/button[2]')))
            btn_modal.click()

        except Exception as e:
            if driver.find_elements(By.XPATH, '//*[@id="toast-notification-c38fb503-75e0-4a27-a737-22afe3841796"]'):
                status = "Erro: Notificação de erro detectada"
            else:
                status = f"Erro: {str(e)}"

        # Registrando a ação no banco de dados SQLite
        data_hora = time.strftime("%Y-%m-%d %H:%M:%S")
        c.execute("INSERT INTO registros (CNPJ, Razao_Social, Valor, Titulo, Status, Data_Hora) VALUES (?, ?, ?, ?, ?, ?)",
                  (str(row['CNPJ/CPF']), row['Nome cliente'], row['Valor'], f"{documento}/{row['Parcela']}", status, data_hora))
        conn.commit()

    # Iterando sobre cada linha do DataFrame para automação
    for index, row in df.iterrows():
        if pd.notna(row['CNPJ/CPF']):  # Verificando se CNPJ/CPF não é nulo
            automat_negative(row)  # Chamando função para automatizar a negativação
            time.sleep(3)  # Aguardando 3 segundos entre cada ação

    driver.close()  # Fechando o navegador Selenium
    conn.close()  # Fechando conexão com o banco de dados SQLite
    messagebox.showinfo("Informação", "Processo concluído.")  # Exibindo mensagem de conclusão

except Exception as e:
    driver.quit()  # Encerrando navegador Selenium em caso de erro
    messagebox.showerror("Erro", f"Ocorreu um erro: {str(e)}")  # Exibindo mensagem de erro
```

3.3 Consulta e Exportação de Dados

3.3.1 *Objetivo*

- Consultar e exportar os dados de negativação registrados para um arquivo Excel.

3.3.2 *Componentes:*

- Função consultar_dados()
- exportar_dados():

3.3.3 *Código*:

```python
# Função para consultar e exibir dados do banco de dados SQLite na interface
def consultar_dados():
    conn = sqlite3.connect('negativacao.db')
    df = pd.read_sql_query("SELECT * FROM registros", conn)
    conn.close()

    # Limpando a visualização atual na árvore de dados
    for i in tree.get_children():
        tree.delete(i)

    # Inserindo novos dados na árvore de dados (TreeView) na interface
    for index, row in df.iterrows():
        tree.insert("", "end", values=(row['CNPJ'], row['Razao_Social'], row['Valor'], row['Titulo'], row['Status'], row['Data_Hora']))

# Função para exportar dados do banco de dados SQLite para um arquivo Excel
def exportar_dados():
    conn = sqlite3.connect('negativacao.db')
    df = pd.read_sql_query("SELECT * FROM registros", conn)
    conn.close()

    # Solicitando ao usuário o local para salvar o arquivo Excel exportado
    file_path = filedialog.asksaveasfilename(defaultextension=".xlsx", filetypes=[("Excel files", "*.xlsx")])
    if file_path:
        df.to_excel(file_path, index=False)  # Exportando DataFrame para o arquivo Excel
        messagebox.showinfo("Informação", "Dados exportados com sucesso.")  # Exibindo mensagem de sucesso
```

4 Conclusão

O código apresentado é uma solução robusta e integrada para automatização de processos de negativação de registros utilizando Python. Através de uma interface gráfica intuitiva construída com tkinter, o usuário pode facilmente realizar login, importar dados de um arquivo Excel, e automatizar o preenchimento de formulários em um site utilizando Selenium. Os registros de cada ação são armazenados localmente em um banco de dados SQLite, permitindo consulta e exportação posterior dos dados. Essa aplicação não só simplifica o processo de negativação, mas também demonstra a eficiência e versatilidade das ferramentas Python para automação e gerenciamento de dados.